

Centro de Estudios Profesionales Santa Gema

Desarrollo de Aplicaciones Multiplataforma

CineAI

Memoria del Proyecto de Fin de Ciclo

Plataforma móvil de recomendaciones cinematográficas mediante inteligencia artificial

Alumno: Héctor Grande Sánchez

Tutor: David Melo

Ciclo: DAM | Curso: 2024-2026 | Fecha: junio de 2026

Resumen

CineAI es una aplicación móvil desarrollada con Flutter para plataformas Android que integra un sistema de recomendaciones cinematográficas basado en inteligencia artificial. La aplicación permite a los usuarios descubrir, organizar y valorar películas, incorporando un asistente conversacional llamado CineBot que utiliza el modelo de lenguaje GPT-4o-mini de OpenAI para ofrecer recomendaciones personalizadas en lenguaje natural. El sistema hace uso de la API pública de The Movie Database (TMDb) como fuente de datos cinematográficos, Firebase Authentication para la gestión de identidades de usuario, Cloud Firestore como base de datos en la nube y Firebase Cloud Messaging para el envío de notificaciones push. La interfaz sigue las directrices de Material Design 3 con una estética oscura inspirada en Letterboxd y Netflix. El resultado es una aplicación completamente funcional compuesta por 15 pantallas, 22 archivos Dart y aproximadamente 6.000 líneas de código, compilada y probada tanto en emulador Android (Pixel 6, API 34) como en dispositivo físico (Xiaomi Redmi Note 5, Android 9).

Palabras clave: Flutter, Dart, Firebase, inteligencia artificial, TMDb, OpenAI, recomendación de películas, aplicación móvil, Android.

Abstract

CineAI is a mobile application developed with Flutter for Android that integrates an AI-based movie recommendation system. The app allows users to discover, organize and rate films through a conversational assistant called CineBot, powered by OpenAI's GPT-4o-mini model. It uses the TMDb API as its cinematic data source, Firebase Authentication for user management, Cloud Firestore as the cloud database and Firebase Cloud Messaging for push notifications. The result is a fully functional 15-screen app tested on both emulator and physical Android device.

Keywords: Flutter, Dart, Firebase, artificial intelligence, TMDb, OpenAI, movie recommendation, mobile application, Android.

Resumen	2
Abstract	2
1. Introducción	4
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos específicos	5
3. Justificación del proyecto	6
3.1. Motivación y contexto de mercado	6
3.2. Elección del stack tecnológico	6
4. Marco teórico	8
4.1. Flutter y Dart	8
4.2. Firebase Authentication	8
4.3. Cloud Firestore	9
4.4. Firebase Cloud Messaging	9
4.5. The Movie Database API (TMDb)	9
4.6. OpenAI API y GPT-4o-mini	9
4.7. Material Design 3	10
5. Metodología	11
5.1. Desarrollo iterativo por fases	11
5.2. Planificación temporal	11
6. Análisis de requisitos	13
6.1. Requisitos funcionales	13
6.2. Requisitos no funcionales	14
6.3. Casos de uso	14
7. Diseño del sistema	16
7.1. Arquitectura general	16
7.2. Modelo de datos en Cloud Firestore	16
7.3. Diagrama de flujo de autenticación	16
7.4. Diagrama de navegación entre pantallas	17
8. Codificación	18
8.1. Estructura de ficheros del proyecto	18
8.2. Módulo de autenticación (servicio_auth.dart)	18
8.3. Módulo TMDb (servicio_tmdb.dart)	18
8.4. Módulo Firebase (servicio_firebase.dart)	19
8.5. Módulo OpenAI y CineBot (servicio_openai.dart)	19
8.6. Módulo de notificaciones (servicio_notificaciones.dart)	19
8.7. Pantalla de Inicio (pantalla_inicio.dart)	19
8.8. Pantalla de Detalle (pantalla_detalle.dart)	19
8.9. CineBot (pantalla_chat.dart)	20
9. Análisis legal y seguridad	21
9.1. Privacidad de datos y RGPD	21
9.2. Seguridad de las credenciales de API	21
9.3. Uso de APIs de terceros	21
10. Pruebas	22

10.1. Pruebas de caja negra	22
10.2. Pruebas de caja blanca	23
11. Organización del proyecto	25
11.1. Metodología y control de versiones	25
11.2. Herramientas utilizadas	25
11.3. Métricas del proyecto	25
12. Conclusiones	26
12.1. Objetivos alcanzados	26
12.2. Dificultades encontradas y soluciones	26
12.3. Líneas de trabajo futuras	26
13. Bibliografía	27
Anexos	28
Anexo A — Estructura completa de archivos	28
Anexo B — Estructura de colecciones en Cloud Firestore	28
Anexo C — Endpoints TMDb utilizados	28
Anexo D — Géneros TMDb	28
Anexo E — System prompt de CineBot	28
Anexo F — Requisitos mínimos del sistema	29
Anexo G — Métricas de calidad del código	29

1. Introducción

El sector del entretenimiento digital ha experimentado en la última década una transformación radical impulsada por la proliferación de plataformas de streaming. Servicios como Netflix, HBO Max, Amazon Prime Video o Disney+ han puesto a disposición de los usuarios catálogos de decenas de miles de títulos accesibles desde cualquier dispositivo. Sin embargo, esta abundancia ha generado un fenómeno paradójico conocido como la "parálisis por elección": los usuarios invierten tiempo desproporcionado en decidir qué ver, llegando en muchos casos a no ver nada (1).

Las soluciones actuales presentan limitaciones claras. Los algoritmos de recomendación de las grandes plataformas están sesgados hacia su propio catálogo, sin cubrir películas disponibles en otras plataformas o fuera del streaming. Letterboxd, la red social cinematográfica más popular, carece de un asistente de inteligencia artificial conversacional. Las apps de catálogo general no permiten interactuar en lenguaje natural para afinar búsquedas según el estado de ánimo del usuario.

CineAI nace como respuesta a este problema: una aplicación móvil para Android que combina catálogo cinematográfico en tiempo real procedente de TMDb, herramientas completas de organización personal (favoritas, vistas, pendientes, listas personalizadas, valoraciones y reseñas) y un asistente conversacional de IA —CineBot— capaz de sugerir películas según el estado de ánimo, género preferido o director favorito del usuario, mostrando tarjetas interactivas con cartel y puntuación directamente en el chat.

El proyecto ha sido desarrollado por Héctor Grande Sánchez como Trabajo de Fin de Ciclo del grado en Desarrollo de Aplicaciones Multiplataforma (DAM), bajo la tutela de David Melo, en el Centro de Estudios Profesionales Santa Gema durante el curso 2024-2026. Comprende 18 semanas de desarrollo iterativo organizado en tres fases, más una fase de documentación, pruebas y entrega.

Desde el punto de vista técnico, CineAI integra cuatro servicios externos de forma coordinada: la API REST de TMDb para datos de películas, la API de OpenAI para el chat con IA, el ecosistema Firebase (Authentication + Cloud Firestore + Cloud Messaging) para backend sin servidor, y Google Fonts para tipografía. Todo ello construido sobre Flutter 3.x con Dart 3.x, siguiendo Material Design 3 y una arquitectura limpia en tres capas.

6:53



2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación móvil multiplataforma para Android que permita descubrir, organizar y valorar películas, con un asistente conversacional de IA para recomendaciones personalizadas, cumpliendo los estándares de calidad, usabilidad y seguridad exigidos en un entorno de producción real.

2.2. Objetivos específicos

1. Implementar autenticación segura con Firebase Authentication mediante correo electrónico y contraseña, con persistencia automática de sesión y mensajes de error localizados en español.
2. Integrar la API REST de TMDb para obtener películas en tendencia, populares y recomendadas, con búsqueda por texto libre y filtros avanzados combinables por género, año y puntuación mínima.
3. Desarrollar CineBot, el asistente conversacional de IA basado en GPT-4o-mini, con historial completo de conversación, chips de sugerencias rápidas, indicador de escritura animado y tarjetas interactivas de películas que permiten navegar al detalle.
4. Construir un sistema completo de colecciones personales: favoritas, vistas, pendientes, listas personalizadas con nombre/emoji/color, valoraciones de 1 a 5 estrellas y reseñas de texto libre.
5. Implementar un ranking comunitario global con las 10 películas mejor valoradas por el conjunto de usuarios, calculado en tiempo real sobre Cloud Firestore.
6. Integrar notificaciones push y locales con Firebase Cloud Messaging y flutter_local_notifications, con visibilidad heads-up dentro y fuera de la app.
7. Lograr una experiencia de usuario fluida mediante Material Design 3, tema oscuro personalizado, fuente Nunito, IndexedStack para preservar el estado entre pestañas y mecanismo de deshacer de 4 segundos en todas las eliminaciones.
8. Garantizar el funcionamiento correcto en emulador Android API 34 y en dispositivo físico con Android 9, cubriendo las versiones de Android más utilizadas en España.

3. Justificación del proyecto

3.1. Motivación y contexto de mercado

La motivación principal de CineAI surge de la observación del mercado: existe un hueco entre las aplicaciones de catálogo puro (IMDb, FilmAffinity) y los asistentes de IA de propósito general (ChatGPT). Ninguna aplicación móvil combina hoy en día un catálogo cinematográfico actualizado en tiempo real con un asistente conversacional especializado exclusivamente en películas que muestre tarjetas visuales interactivas directamente en el chat.

Desde el punto de vista académico, el proyecto exige la integración simultánea de cuatro servicios externos con naturalezas muy diferentes —una API REST pública, un BaaS, un LLM y un sistema de notificaciones push—, lo que implica conocimientos sólidos de arquitectura de software, programación asíncrona, gestión del estado y diseño de interfaz de usuario. Estos son precisamente los contenidos transversales del ciclo DAM.

3.2. Elección del stack tecnológico

Flutter frente a React Native o desarrollo nativo: Flutter compila a código ARM nativo mediante AOT, lo que garantiza rendimiento cercano al nativo. Su motor gráfico (Skia/Impeller) dibuja cada píxel directamente, evitando la capa de traducción de widgets que limita a React Native. Además, el ecosistema Firebase tiene soporte oficial de primer nivel para Flutter, lo que simplifica la integración con Authentication, Firestore y FCM.

Firebase frente a un backend propio (Node.js/Django + PostgreSQL): un backend propio requiere configuración de servidor, gestión de escalado, mantenimiento de actualizaciones de seguridad y coste de hosting. Firebase, como plataforma BaaS (Backend as a Service), elimina toda esa infraestructura. El plan Spark (gratuito) cubre sobradamente las necesidades de un TFG: 1 GB de almacenamiento en Firestore, 10.000 autenticaciones/mes y 500.000 lecturas/día (2).

TMDb frente a otras APIs de cine: TMDb es la base de datos cinematográfica más completa disponible de forma gratuita, con más de 500.000 películas, soporte multiidioma (incluyendo español) y endpoints especializados como /discover para filtros avanzados y /recommendations para sugerencias basadas en una película concreta (3). Sus competidores (IMDb API, OMDb) son de pago o tienen limitaciones severas en el plan gratuito.

GPT-4o-mini frente a otros modelos de IA: GPT-4o-mini ofrece el mejor equilibrio entre calidad de respuesta, velocidad de inferencia y coste por token para aplicaciones conversacionales de usuario final. Su contexto de 128.000 tokens permite mantener el historial completo de la conversación sin necesidad de truncamiento. Modelos más

grandes como GPT-4o tienen un coste por token 15 veces mayor sin aportar una mejora proporcional en el caso de uso de recomendaciones de películas (4).

4. Marco teórico

4.1. Flutter y Dart

Flutter es un framework de código abierto desarrollado por Google y publicado en versión estable en diciembre de 2018. En su versión 3.x ofrece soporte para Android, iOS, Web, Windows, macOS y Linux desde una única base de código Dart (11). A diferencia de frameworks como React Native, Flutter no utiliza componentes nativos de la plataforma destino, sino que dibuja cada widget directamente sobre el lienzo gráfico mediante su motor Skia (en dispositivos más antiguos) o Impeller (el nuevo motor, por defecto desde Flutter 3.10), garantizando una apariencia y comportamiento perfectamente consistentes entre plataformas.

Dart es el lenguaje de programación de Flutter. Es orientado a objetos, fuertemente tipado y con inferencia de tipos, lo que reduce la verbosidad del código. Ofrece dos modos de compilación: JIT (Just-In-Time) durante el desarrollo, que habilita el hot reload (recarga de cambios en milisegundos sin perder el estado), y AOT (Ahead-Of-Time) para producción, que genera binarios nativos de máximo rendimiento (12).

Conceptos de Flutter aplicados en CineAI:

- **Widget:** la unidad básica de composición en Flutter. Todo elemento visual es un widget inmutable; cuando el estado cambia, Flutter reconstruye el subárbol de widgets afectado.
- **StatelessWidget:** widget sin estado interno mutable. Usado en CineAI para TarjetaPelicula y otros componentes puramente visuales.
- **StatefulWidget:** widget con estado interno que puede cambiar durante el ciclo de vida. Usado en todas las pantallas que manejan datos variables.
- **StreamBuilder:** widget que escucha un Stream y reconstruye su subárbol con cada nuevo valor. En main.dart escucha `authStateChanges()` para navegar automáticamente entre login y la app.
- **IndexedStack:** mantiene todos sus hijos en memoria aunque solo muestre uno. Usado en PantallaShell para los 5 tabs, preservando el estado al cambiar de pestaña.
- **Future y async/await:** modelo de concurrencia para operaciones asíncronas. `Future.wait([])` lanza múltiples Futures en paralelo, reduciendo la latencia total.

4.2. Firebase Authentication

Firebase Authentication es el servicio de gestión de identidades de la plataforma Firebase, desarrollada por Google (5). Proporciona autenticación mediante

correo/contraseña, teléfono y proveedores de identidad federada. En CineAI se usa exclusivamente correo/contraseña.

Características técnicas relevantes: persistencia de sesión automática mediante JWT almacenado en el dispositivo; UID único de 28 caracteres como clave primaria en Firestore; códigos de error en inglés traducidos al español por `ServicioAuth.traducirError()`; `updateDisplayName()` para actualizar el nombre sin campos adicionales en Firestore.

4.3. Cloud Firestore

Cloud Firestore es la base de datos NoSQL orientada a documentos de Firebase, lanzada en 2017 como sucesor de Firebase Realtime Database (6). Los datos se organizan en colecciones de documentos sin esquema predefinido (schema-less), lo que permite almacenar estructuras de datos heterogéneas sin migraciones.

Características clave: jerarquía colección→documento→subcolección; lecturas offline con caché en disco; `set()` con `merge:true` para actualizar campos concretos sin sobrescribir el documento; `FieldValue.delete()` para eliminar campos individuales; `FieldValue.arrayUnion()` para añadir elementos a arrays; `serverTimestamp()` para marcas de tiempo del servidor.

4.4. Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) es el servicio de mensajería push de Google, sucesor de Google Cloud Messaging (7). Permite enviar mensajes a dispositivos Android, iOS y Web de forma gratuita y sin límite de mensajes.

CineAI combina `firebase_messaging` (notificaciones FCM en segundo plano) con `flutter_local_notifications` (notificaciones locales en primer plano, con `Importance.max` + `Priority.max` para visibilidad heads-up). El canal Android "cineai_canal" se registra en el primer arranque con importancia máxima, sonido y badge.

4.5. The Movie Database API (TMDb)

The Movie Database (TMDb) es una base de datos cinematográfica fundada en 2008, con más de 500.000 películas indexadas (8). Su API REST v3 es gratuita sin límite de peticiones para proyectos no comerciales.

Características usadas: `language=es-ES` para contenido en español; `append_to_response=credits` para obtener dirección y reparto en una sola petición; `vote_count.gte=50` en `/discover` para filtrar películas sin votos suficientes.

4.6. OpenAI API y GPT-4o-mini

OpenAI pone a disposición una API REST para acceder a sus modelos LLM mediante el endpoint `/v1/chat/completions` (9). La API acepta mensajes con roles `system`, `user` y `assistant`. GPT-4o-mini, lanzado en julio de 2024, ofrece ventana de contexto de 128.000 tokens a un coste aproximadamente 15 veces menor que GPT-4o, con calidad suficiente para recomendaciones cinematográficas.

Prompt engineering en CineAI: el system prompt de CineBot define tono, idioma, ámbito (solo películas), formato (3-5 recomendaciones con frase + pregunta de cierre) y el bloque especial `CINEAI_PELICULAS:[...]` que la app parsea para mostrar tarjetas. Temperature 0.8 añade variedad sin sacrificar coherencia.

4.7. Material Design 3

Material Design 3 (Material You) es el sistema de diseño de Google publicado en 2021 (10). CineAI aplica: fondo #0D0D14, superficie #161625, azul primario #2E75B6, dorado secundario #FFD700 para estrellas; fuente Nunito de Google Fonts con peso 800 en títulos; tarjetas con BorderRadius 16dp y elevación 4dp; áreas de toque mínimas de 48x48dp.

5. Metodología

5.1. Desarrollo iterativo por fases

CineAI se ha desarrollado siguiendo una metodología iterativa e incremental inspirada en Scrum, adaptada al contexto de un desarrollador individual. Se definieron tres iteraciones de seis semanas cada una, con objetivos claros al inicio y revisión al final. Esta metodología permite entregar valor funcional de forma continua y ajustar el alcance si surgen imprevistos técnicos.

Iteración 1 — Núcleo cinematográfico (semanas 1-6): app funcional con datos reales de TMDb, sin autenticación. Entregable: PantallaInicio con banner, tendencias y populares; PantallaDetalle; TarjetaPelicula reutilizable; navegación básica.

Iteración 2 — Inteligencia artificial y autenticación (semanas 7-12): integración de Firebase Auth y CineBot. Entregable: flujo auth con StreamBuilder; CineBot completo con historial, tarjetas y notificaciones; filtros /discover; tema oscuro definitivo con Nunito.

Iteración 3 — Funcionalidades sociales y datos personales (semanas 13-18): ecosistema completo en Firestore. Entregable: favoritas, vistas, pendientes con caché local; valoraciones y reseñas; listas personalizadas; ranking comunitario; mecanismo de deshacer; PantallaPerfil.

5.2. Planificación temporal

Tabla 1. Planificación temporal del proyecto CineAI

Semanas	Fase	Tareas principales
1-2	Análisis y diseño	Requisitos, tecnologías, arquitectura, estructura Firestore
3-4	Iter. 1 — Inicio	ServicioTMDb, PantallaInicio, TarjetaPelicula, banner
5-6	Iter. 1 — Detalle	PantallaDetalle, navegación, barra inferior, IndexedStack
7-8	Iter. 2 — Auth	Firebase Auth, ServicioAuth, PantallaLogin, PantallaRegistro
9-10	Iter. 2 — IA	ServicioOpenAI, PantallaChat, CineBot, parsearRespuesta
11-12	Iter. 2 — UI	Tema oscuro, Nunito, notificaciones FCM, filtros /discover
13-14	Iter. 3 — CRUD	ServicioFirestore, caché local, favoritas, vistas, pendientes

15-16	Iter. 3 — Social	Valoraciones, reseñas, ranking, listas, deshacer
17	Pruebas	Caja negra y blanca, dispositivo físico Xiaomi Redmi Note 5
18	Documentación	Memoria TFG, manuales, APK release

6. Análisis de requisitos

6.1. Requisitos funcionales

RF-01. El sistema permitirá registrar usuarios mediante correo electrónico y contraseña, validando el formato del correo y que la contraseña tenga al menos 6 caracteres.

RF-02. El sistema mantendrá la sesión activa entre reinicios de la app y navegará automáticamente a la pantalla principal sin requerir login repetido.

RF-03. La pantalla de inicio mostrará un saludo personalizado con el nombre del usuario y el momento del día (buenos días / tardes / noches).

RF-04. La pantalla de inicio incluirá un banner deslizante automático con las 7 primeras películas en tendencia, con cambio de página cada 4 segundos.

RF-05. La pantalla de inicio mostrará tres secciones: "Recomendadas para ti" (basadas en favoritas), "Tendencias hoy" y "Más populares".

RF-06. El usuario podrá buscar películas por título mediante una barra de búsqueda.

RF-07. El usuario podrá aplicar filtros avanzados combinables: género (10 géneros), año mínimo de estreno y puntuación mínima de 1 a 9.

RF-08. La pantalla de detalle mostrará: cartel, título, sinopsis, año, duración, géneros, idioma original, puntuación TMDb, director y los 6 primeros actores del reparto.

RF-09. El usuario podrá marcar/desmarcar cada película como favorita, vista o pendiente desde la pantalla de detalle.

RF-10. Al eliminar una película de cualquier colección, el sistema mostrará un SnackBar con botón "Deshacer" activo durante 4 segundos.

RF-11. El usuario podrá valorar cada película con 1 a 5 estrellas. Si pulsa la misma puntuación, se eliminará la valoración.

RF-12. El usuario podrá escribir y guardar una reseña de texto libre por película. Si el texto está vacío, la reseña se elimina.

RF-13. El usuario podrá crear listas personalizadas con nombre, emoji y color, y añadir/eliminar películas de cada lista.

RF-14. CineBot aceptará mensajes en lenguaje natural y responderá en español con 3-5 recomendaciones de películas, mostrando tarjetas interactivas.

RF-15. CineBot mostrará chips de sugerencias rápidas al inicio de la conversación.

RF-16. La pantalla de Actividad mostrará estadísticas del usuario y el ranking comunitario con las 10 películas mejor valoradas.

RF-17. El sistema enviará notificaciones heads-up en tres situaciones: al cargar el inicio, al recibir recomendaciones de CineBot y al pulsar "Probar notificación" en el perfil.

RF-18. El usuario podrá editar su nombre desde la pantalla de perfil.

RF-19. El usuario podrá cerrar sesión desde la pantalla de perfil.

6.2. Requisitos no funcionales

RNF-01. Rendimiento: carga de inicio en menos de 3 segundos bajo WiFi estándar, gracias a llamadas paralelas con `Future.wait()` y caché local.

RNF-02. Usabilidad: elementos interactivos con área de toque mínima de 48x48dp. Texto mínimo de 12sp.

RNF-03. Compatibilidad: Android API 21 (Android 5.0) o superior, cubriendo más del 99% de dispositivos activos.

RNF-04. Seguridad: credenciales solo en Firebase Authentication, con contraseñas hasheadas por Firebase. Sin contraseñas en local ni en Firestore.

RNF-05. Disponibilidad: todos los errores muestran mensajes en español sin cierre inesperado. Todos los bloques asíncronos envueltos en `try/catch`.

RNF-06. Mantenibilidad: código en tres capas (pantallas, servicios, modelos) con nombres en español siguiendo convenciones Dart.

RNF-07. Internacionalización: datos TMDb con `language=es-ES`. Errores Firebase traducidos al español.

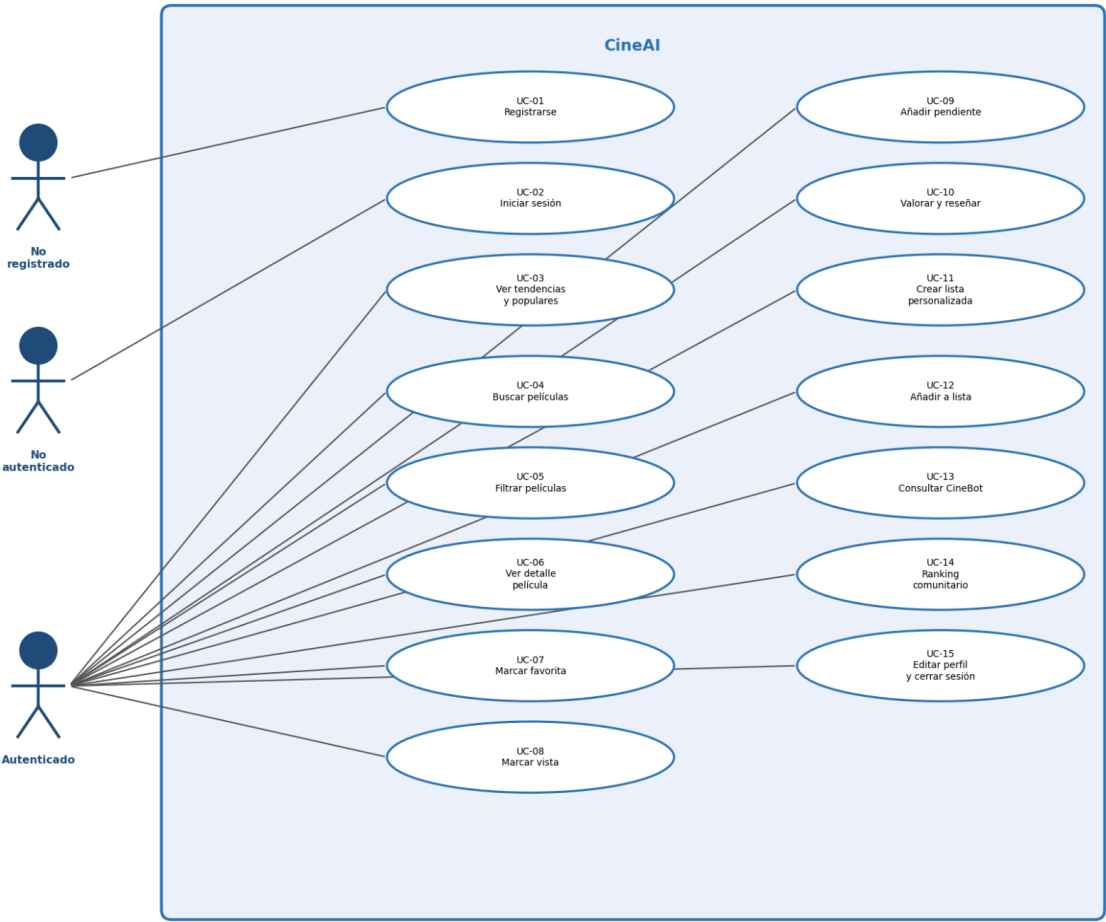
6.3. Casos de uso

Tabla 2. Casos de uso del sistema CineAI

UC	Actor	Precondición	Descripción
UC-01	No registrado	Ninguna	Registrarse con email y contraseña
UC-02	No autenticado	Cuenta creada	Iniciar sesión en la app
UC-03	Autenticado	Sesión activa	Ver películas en tendencia y populares
UC-04	Autenticado	Sesión activa	Buscar películas por título
UC-05	Autenticado	Sesión activa	Filtrar por género, año y/o puntuación
UC-06	Autenticado	Sesión activa	Ver detalle completo de una película

UC-07	Autenticado	En detalle	Marcar/desmarcar como favorita
UC-08	Autenticado	En detalle	Marcar/desmarcar como vista
UC-09	Autenticado	En detalle	Añadir/quitar de pendientes
UC-10	Autenticado	En detalle	Valorar con estrellas y escribir reseña
UC-11	Autenticado	Sesión activa	Crear lista personalizada
UC-12	Autenticado	Lista creada	Añadir película a lista personalizada
UC-13	Autenticado	Sesión activa	Consultar CineBot y recibir recomendaciones
UC-14	Autenticado	Sesión activa	Consultar ranking comunitario
UC-15	Autenticado	Sesión activa	Editar nombre y/o cerrar sesión

Diagrama de Casos de Uso — CineAI

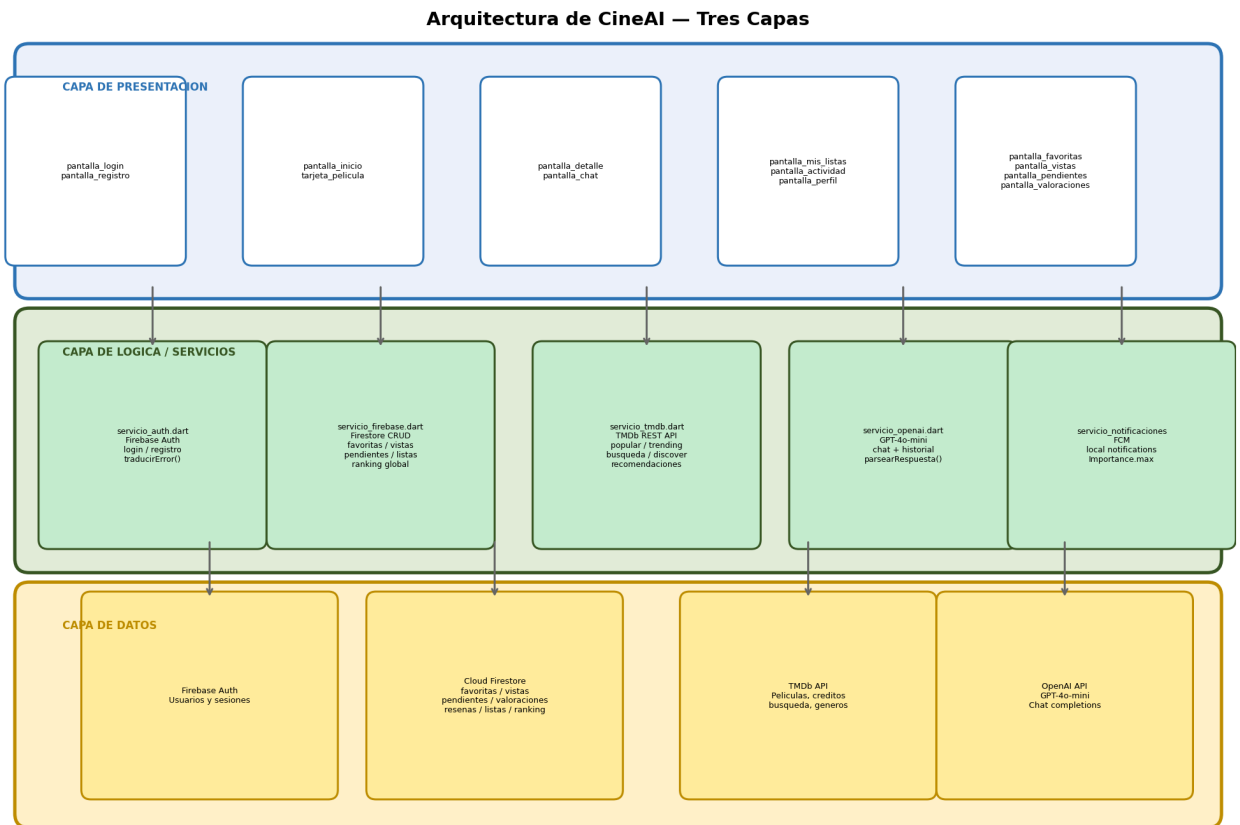


7. Diseño del sistema

7.1. Arquitectura general

CineAI sigue una arquitectura de tres capas bien diferenciadas:

- Capa de presentación (pantallas y componentes): 15 StatefulWidget/StatelessWidget que muestran datos y capturan interacciones. No contienen lógica de negocio ni acceden directamente a APIs externas.
- Capa de servicios (lógica de negocio): cinco clases con responsabilidades definidas — ServicioAuth, ServicioTMDb, ServicioFirebase, ServicioOpenAI y ServicioNotificaciones.
- Capa de modelos (entidades de datos): la clase Pelicula centraliza la representación de una película. Sus métodos estáticos de factoría encapsulan la transformación del formato externo al modelo interno.



7.2. Modelo de datos en Cloud Firestore

La estructura de Firestore está diseñada para minimizar lecturas y maximizar el rendimiento de las operaciones más frecuentes.

Colecciones por usuario (usuarios/{uid}/...): favoritas/{movieId}, vistas/{movieId}, pendientes/{movieId} — datos completos de la película. valoraciones/{movieId} —

puntuación 1-5 y fechaValoracion. `resenas/{movieId}` — texto y fecha. `listas_personalizadas/{listId}` — nombre, emoji, color, array de películas.

Colección global `ranking/{movieId}`: estructura `{votos: {uid1: 4, uid2: 5, ...}}`. Un documento por película con el mapa de votos de todos los usuarios. `merge:true` garantiza que cada usuario solo actualice su propio campo.

Caché local en memoria: tres `Set<int>` (`_idsFavoritas`, `_idsVistas`, `_idsPendientes`) y un `Map<int, int>` de valoraciones propias cargados en paralelo al arrancar. Las consultas son síncronas $O(1)$, sin llamadas a Firestore en cada render de tarjeta.

7.3. Diagrama de flujo de autenticación

El `StreamBuilder` en `main.dart` escucha `authStateChanges()` de Firebase Auth. Al arrancar: `Firebase.initializeApp()` → `ServicioNotificaciones().inicializar()` → `StreamBuilder`. Si el stream emite `User` → `PantallaShell`; si emite `null` → `PantallaLogin`; si `ConnectionState.waiting` → `_PantallaCarga`.

Al hacer login: `PantallaLogin` llama a `ServicioAuth.iniciarSesion()` → Firebase Auth actualiza el stream → `StreamBuilder` muestra `PantallaShell` automáticamente. Al cerrar sesión: el flujo es el inverso.

7.4. Diagrama de navegación entre pantallas

Tabla 3. Rutas de navegación del sistema

Origen	Destino	Acción
<code>main.dart</code>	<code>PantallaLogin</code>	Usuario no autenticado al arrancar
<code>main.dart</code>	<code>PantallaShell</code>	Usuario autenticado al arrancar o tras login
<code>PantallaLogin</code>	<code>PantallaRegistro</code>	Tap en "Crear cuenta"
<code>PantallaRegistro</code>	<code>PantallaShell</code>	Registro exitoso
Shell tab 0	<code>PantallaInicio</code>	Activa por defecto
Shell tab 1	<code>PantallaMisListas</code>	Tap en icono listas
Shell tab 2	<code>PantallaChat</code>	Tap en icono CineBot
Shell tab 3	<code>PantallaActividad</code>	Tap en icono actividad
Shell tab 4	<code>PantallaPerfil</code>	Tap en icono perfil
<code>PantallaInicio</code>	<code>PantallaDetalle</code>	Tap en tarjeta o banner
<code>PantallaChat</code>	<code>PantallaDetalle</code>	Tap en tarjeta recomendada
<code>PantallaMisListas</code>	<code>PantallaFavoritas/Vistas/Pendientes</code>	Tap en opción correspondiente

PantallaActividad	PantallaDetalle	Tap en película del ranking
PantallaValoraciones	PantallaDetalle	Tap en película valorada
PantallaResenas	PantallaDetalle	Tap en película reseñada

8. Codificación

8.1. Estructura de ficheros del proyecto

El proyecto CineAI contiene 22 archivos Dart organizados en tres carpetas bajo lib/:

lib/ main.dart — Entry point, tema global, StreamBuilder de auth

lib/modelos/ pelicula.dart — Clase Pelicula con Factory Methods

lib/servicios/ servicio_auth.dart | servicio_firebase.dart | servicio_tmdb.dart |
servicio_openai.dart | servicio_notificaciones.dart

lib/pantallas/ 15 pantallas: shell, login, registro, inicio, detalle, chat, mis_listas,
lista_personalizada, actividad, perfil, favoritas, vistas, pendientes, valoraciones, resenas

lib/componentes/ tarjeta_pelicula.dart — Widget reutilizable

8.2. Módulo de autenticación (servicio_auth.dart)

ServicioAuth encapsula la interacción con Firebase Auth en cuatro métodos públicos: iniciarSesion(), registrarse(), cerrarSesion() y traducirError(). El método registrarse() llama a createUserWithEmailAndPassword() y seguidamente a updateDisplayName() en la misma llamada, de modo que el nombre del usuario está disponible desde el primer login sin necesidad de un documento adicional en Firestore.

El método traducirError() convierte los códigos de error de Firebase (en inglés) en mensajes comprensibles en español mediante un switch sobre los códigos más comunes: user-not-found, wrong-password, invalid-credential, email-already-in-use, weak-password, invalid-email, too-many-requests y network-request-failed.

8.3. Módulo TMDb (servicio_tmdb.dart)

ServicioTMDb implementa un cliente HTTP sobre el paquete http de Dart. En caso de error de red o código distinto de 200, los métodos devuelven lista vacía o null, garantizando RNF-05.

El método obtenerPopularesAleatorios() selecciona dos páginas aleatorias de /movie/popular, las descarga en paralelo con Future.wait(), las combina y mezcla con Random(), haciendo que la sección "Más populares" muestre contenido diferente en cada sesión.

El método buscarPorTituloAnio() es específico para CineBot: busca sin el parámetro language para encontrar películas por título original. Si la búsqueda con año no devuelve resultados, reintenta sin el filtro de año como fallback.

El método `descubrirPelículas()` construye dinámicamente los parámetros de la URL solo con los filtros especificados, añadiendo `vote_count.gte=50` cuando se filtra por puntuación.

8.4. Módulo Firebase (`servicio_firebase.dart`)

`ServicioFirebase` es la clase más extensa del proyecto (~470 líneas). El método `inicializar()` lanza en paralelo la carga de los cuatro conjuntos de datos mediante `Future.wait()`. Las operaciones de escritura actualizan primero la caché local y luego persisten en Firestore de forma asíncrona, garantizando feedback visual inmediato.

El mecanismo de deshacer se implementa en las pantallas: 1) eliminar de caché local y UI inmediatamente; 2) mostrar `SnackBar` con botón "Deshacer"; 3) si el usuario pulsa antes de 4 segundos, cancelar la eliminación de Firestore; si no, ejecutar la eliminación definitiva.

El método `obtenerTopRanking()` lee la colección global `ranking`, calcula la media aritmética de los votos por película y devuelve las 10 con mayor media, redondeada a un decimal.

8.5. Módulo OpenAI y CineBot (`servicio_openai.dart`)

El método `enviarMensaje()` antepone siempre el `system prompt` al historial de conversación. La petición usa `max_tokens: 500` y `temperature: 0.8`. La decodificación usa `utf8.decode(bodyBytes)` para preservar tildes y caracteres Unicode.

El método estático `parsearRespuesta()` localiza la marca `CINEAI_PELICULAS:`, extrae el array JSON usando `indexOf('[')` y `lastIndexOf(']')` — robusto ante texto extra que GPT a veces añade tras el JSON — y devuelve la tupla (`textoLimpio`, `listaDePelículas`).

8.6. Módulo de notificaciones (`servicio_notificaciones.dart`)

`ServicioNotificaciones` implementa el patrón Singleton mediante un constructor `factory`. El canal Android `"cineai_canal"` se crea con `Importance.max` para notificaciones `heads-up`. La función `_manejarMensajeFondo()` es top-level marcada con `@pragma('vm:entry-point')` para que el compilador AOT no la elimine.

8.7. Pantalla de Inicio (`pantalla_inicio.dart`)

`PantallaInicio` es la pantalla más compleja de CineAI. Contiene: banner `PageView` con 7 películas trending y Timer de 4 segundos; saludo personalizado según hora del día y nombre del usuario; sección "Recomendadas para ti" basada en favoritas del usuario vía `/recommendations`; filtros avanzados expandibles que llaman a `/discover`.

8.8. Pantalla de Detalle (`pantalla_detalle.dart`)

PantallaDetalle obtiene el detalle completo mediante `append_to_response=credits` si la tarjeta no trae todos los datos. Los botones de favorita/vista/pendiente consultan la caché local (síncrono $O(1)$) y actualizan UI inmediatamente. El sistema de valoración con 5 `IconButton` implementa comportamiento toggle: pulsar la misma estrella activa elimina la valoración.

8.9. CineBot (`pantalla_chat.dart`)

El historial de mensajes se almacena en `List<MensajeConversacion>` en el estado del widget. Cada mensaje dispara `enviarMensaje(historial)`, se parsea la respuesta y se buscan las películas en TMDb en paralelo con `Future.wait()`. El indicador de escritura (tres puntos animados) aparece mientras se espera la respuesta. `notificarRecomendacionIA()` se llama tras cada respuesta con películas.

9. Análisis legal y seguridad

9.1. Privacidad de datos y RGPD

CineAI almacena en Cloud Firestore: correo electrónico y nombre (gestionados por Firebase Authentication), y preferencias cinematográficas del usuario (favoritas, vistas, pendientes, valoraciones, reseñas y listas personalizadas). Todos estos datos están protegidos bajo la cuenta de Google/Firebase del desarrollador.

De acuerdo con el RGPD (Reglamento (UE) 2016/679), el usuario tiene derecho a acceso, rectificación y supresión de sus datos. En CineAI, el usuario puede modificar su nombre desde el perfil. Los datos se almacenan en servidores de Google en la región europe-west, cumpliendo con el RGPD al mantenerse en territorio de la Unión Europea.

9.2. Seguridad de las credenciales de API

La clave de API de TMDb y el token Bearer de OpenAI se almacenan como constantes en el código. En producción deberían almacenarse en Firebase Remote Config o variables de entorno del servidor. Para el contexto académico de un TFG, esta simplificación es aceptable.

Las reglas de Firestore Security Rules están en modo de prueba (allow read, write: if true). En producción deberían configurarse para que cada usuario solo acceda a sus propios documentos y la colección ranking sea de lectura pública y escritura solo para autenticados.

9.3. Uso de APIs de terceros

TMDb API: uso bajo licencia de API de TMDb, permitido para aplicaciones no comerciales con la condición de mostrar el logotipo oficial (3). OpenAI API: uso bajo términos de servicio de OpenAI; GPT-4o-mini no almacena conversaciones de forma persistente según la política de privacidad de OpenAI para llamadas a API. Firebase: uso bajo el plan Spark gratuito, sujeto a los términos de servicio de Google Cloud Platform (5).

10. Pruebas

10.1. Pruebas de caja negra

Las pruebas de caja negra evalúan el comportamiento externo del sistema desde la perspectiva del usuario, sin conocimiento de la implementación interna.

Tabla 4. Pruebas de caja negra

Caso	Descripción	Entrada	Salida esperada	Resultado
CN-01	Registro con email válido	Email: test@test.com, Pass: 123456, Nombre: Ana	Navegación a PantallaShell con saludo "Hola, Ana"	Correcto
CN-02	Registro con email duplicado	Email ya registrado	Mensaje "Ya existe una cuenta con ese correo."	Correcto
CN-03	Registro con contraseña corta	Contraseña: "123"	Mensaje "La contraseña debe tener al menos 6 caracteres."	Correcto
CN-04	Login con credenciales correctas	Email y contraseña registrados	Navegación a PantallaShell	Correcto
CN-05	Login con contraseña incorrecta	Email correcto, pass incorrecto	Mensaje "Correo o contraseña incorrectos."	Correcto
CN-06	Búsqueda de película existente	Texto: "Inception"	Lista con Inception y resultados relacionados	Correcto
CN-07	Búsqueda de película inexistente	Texto: "xyzxyzxyz123"	Lista vacía con mensaje "Sin resultados"	Correcto
CN-08	Marcar película como favorita	Tap en icono favorita en PantallaDetalle	Icono relleno, película en PantallaFavoritas	Correcto
CN-09	Deshacer eliminación de favorita	Eliminar y pulsar "Deshacer" en < 4s	Película restaurada en PantallaFavoritas	Correcto
CN-10	Valorar con 3 estrellas	Tap en 3a estrella en PantallaDetalle	3 estrellas doradas activas, valoración guardada	Correcto
CN-11	Quitar valoración misma estrella	Tap en estrella activa	0 estrellas activas, valoración eliminada	Correcto

CN-12	CineBot: solicitar acción	Mensaje: "Recomiendame acción"	Respuesta con 3-5 películas y tarjetas	Correcto
CN-13	Filtro por género Terror	Seleccionar "Terror" y Aplicar	Lista de películas de terror de TMDb	Correcto
CN-14	Crear lista personalizada	Nombre, emoji, color azul	Lista creada en PantallaMisListas	Correcto
CN-15	Ver ranking comunitario	Navegar a PantallaActividad	Top 10 con media y cartel	Correcto

10.2. Pruebas de caja blanca

Las pruebas de caja blanca evalúan la lógica interna de los métodos, verificando flujos de ejecución, condiciones de borde y estructuras de datos internas.

Tabla 5. Pruebas de caja blanca

Caso	Método	Condición	Resultado esperado	Resultado
CB-01	parsearRespuesta()	Sin marca CINEAI_PELICULAS	Devuelve (texto completo, lista vacía)	Correcto
CB-02	parsearRespuesta()	JSON válido	Devuelve (texto limpio, lista de películas)	Correcto
CB-03	parsearRespuesta()	Texto extra después del]	Extrae solo el array, ignora texto extra	Correcto
CB-04	traducirError()	Código "user-not-found"	"No existe ninguna cuenta con ese correo."	Correcto
CB-05	traducirError()	Código no contemplado	"Se produjo un error. Inténtalo de nuevo."	Correcto
CB-06	_filtrarIdioma()	Película en árabe ("ar")	Película excluida del resultado	Correcto
CB-07	_filtrarIdioma()	Película en inglés ("en")	Película incluida en el resultado	Correcto
CB-08	inicializar()	Usuario sin favoritas	_idsFavoritas queda como Set vacío, sin error	Correcto
CB-09	estaGuardadaComoFavorita()	ID presente en caché	Devuelve true sin llamada a Firestore	Correcto
CB-10	estaGuardadaComoFavorita()	ID ausente en caché	Devuelve false sin llamada a Firestore	Correcto

CB-11	obtenerTopRanking()	Colección ranking vacía	Devuelve lista vacía sin error	Correcto
CB-12	obtenerTopRanking()	Votos: {uid1:4, uid2:2}	Media = 3.0, incluida en la lista	Correcto
CB-13	guardarResena()	Texto de reseña vacío	Llama a delete() en lugar de set()	Correcto
CB-14	obtenerPopularesAleatorios()	Error de red en ambas páginas	Llama a obtenerPeliculasPopulares() como fallback	Correcto
CB-15	buscarPorTituloAnio()	Sin resultado con año	Reintenta sin el parámetro year	Correcto

11. Organización del proyecto

11.1. Metodología y control de versiones

El proyecto CineAI se desarrolló a lo largo de 18 semanas con control de versiones mediante Git. El repositorio se inicializó en la semana 1 y se han realizado commits frecuentes al finalizar cada funcionalidad significativa, con mensajes descriptivos en español.

La organización del trabajo siguió el principio de "funcionalidad mínima viable": cada característica se implementó de principio a fin (UI + servicio + persistencia) antes de pasar a la siguiente, manteniendo siempre una versión funcional ejecutable en el emulador.

11.2. Herramientas utilizadas

- Flutter SDK 3.x + Dart SDK 3.x: entorno de desarrollo principal.
- Android Studio: IDE con emulador Pixel 6 API 34.
- Firebase Console: gestión de Authentication, Firestore y FCM.
- Git: control de versiones local.
- Google Docs: redacción de la memoria y los manuales.

11.3. Métricas del proyecto

Métrica	Valor
Archivos Dart	22
Líneas de código (aprox.)	6.000
Pantallas	15
Servicios	5
Modelos	1 (clase Pelicula)
APK release	51,9 MB
Endpoints TMDb	6
Semanas de desarrollo	18
Emulador	Pixel 6 API 34 (Android 14)
Dispositivo físico	Xiaomi Redmi Note 5 (Android 9)

12. Conclusiones

12.1. Objetivos alcanzados

Todos los objetivos específicos definidos en el apartado 2.2 han sido implementados y verificados. El objetivo 1 (autenticación) se cumple con ServicioAuth y las pantallas de login y registro. El objetivo 2 (TMDb) se cumple con ServicioTMDb y sus 6 endpoints. El objetivo 3 (CineBot) se cumple con ServicioOpenAI y PantallaChat. El objetivo 4 (colecciones) se cumple con 6 tipos de colecciones en Firestore y caché local. El objetivo 5 (ranking) se cumple con la colección global y PantallaActividad. El objetivo 6 (notificaciones) se cumple con ServicioNotificaciones y tres tipos de notificaciones. El objetivo 7 (UX) se cumple con Material Design 3, IndexedStack y mecanismo de deshacer. El objetivo 8 (compatibilidad) se verifica en emulador Pixel 6 API 34 y dispositivo Xiaomi Redmi Note 5 con Android 9.

12.2. Dificultades encontradas y soluciones

Dificultad 1 — Parseo de respuestas de GPT: GPT-4o-mini no siempre sigue el formato exacto del system prompt. Solución: parser basado en `indexOf('[')` y `lastIndexOf(']')` robusto ante variaciones.

Dificultad 2 — Rendimiento con muchas tarjetas: consultas a Firestore en cada `build()` era inviable. Solución: caché local en memoria `Set<int>` con consultas `O(1)` y síncronas.

Dificultad 3 — Filtros avanzados: filtrar las 40 películas cargadas dejaba la lista vacía. Solución: delegar el filtrado al endpoint `/discover/movie` de TMDb.

Dificultad 4 — Notificaciones heads-up dentro de la app: las notificaciones FCM no se muestran como heads-up en primer plano. Solución: combinar FCM con `flutter_local_notifications (Importance.max)`.

12.3. Líneas de trabajo futuras

- Soporte iOS: la carpeta `ios/` existe; solo requiere Mac con Xcode para compilar.
- Login con Google: añadir Google Sign-In como proveedor federado.
- Publicación en Google Play Store: el APK release está listo.
- Eliminación de cuenta: Cloud Function que elimine usuario de Auth y Firestore en cascada.
- Sincronización en tiempo real: snapshots en lugar de lecturas únicas (`get()`).

13. Bibliografía

1. Schwartz B. The Paradox of Choice: Why More Is Less. Nueva York: Ecco Press; 2004.
2. Firebase [Internet]. Mountain View: Google LLC; 2014 [citado 5 jun 2026]. Firebase Pricing. Disponible en: <https://firebase.google.com/pricing>
3. The Movie Database [Internet]. San Francisco: TMDb; 2008 [citado 5 jun 2026]. TMDb API Documentation. Disponible en: <https://developer.themoviedb.org/docs>
4. OpenAI [Internet]. San Francisco: OpenAI; 2023 [citado 5 jun 2026]. GPT-4o mini: advancing cost-efficient intelligence. Disponible en: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
5. Google Firebase [Internet]. Mountain View: Google LLC; 2014 [citado 5 jun 2026]. Firebase Authentication. Disponible en: <https://firebase.google.com/docs/auth>
6. Google Firebase [Internet]. Mountain View: Google LLC; 2017 [citado 5 jun 2026]. Cloud Firestore Documentation. Disponible en: <https://firebase.google.com/docs/firestore>
7. Google Firebase [Internet]. Mountain View: Google LLC; 2016 [citado 5 jun 2026]. Firebase Cloud Messaging. Disponible en: <https://firebase.google.com/docs/cloud-messaging>
8. The Movie Database [Internet]. San Francisco: TMDb; 2008 [citado 5 jun 2026]. About TMDb. Disponible en: <https://www.themoviedb.org/about>
9. OpenAI [Internet]. San Francisco: OpenAI; 2020 [citado 5 jun 2026]. OpenAI API Reference. Disponible en: <https://platform.openai.com/docs/api-reference>
10. Google [Internet]. Mountain View: Google LLC; 2021 [citado 5 jun 2026]. Material Design 3. Disponible en: <https://m3.material.io>
11. Flutter [Internet]. Mountain View: Google LLC; 2018 [citado 5 jun 2026]. Flutter Documentation. Disponible en: <https://docs.flutter.dev>
12. Dart [Internet]. Mountain View: Google LLC; 2011 [citado 5 jun 2026]. Dart Programming Language. Disponible en: <https://dart.dev/guides>

Anexos

Anexo A — Estructura completa de archivos

Ver sección 8.1 del cuerpo del trabajo.

Anexo B — Estructura de colecciones en Cloud Firestore

usuarios/{uid}/favoritas/{movieId} → identificador, titulo, urlCartel, puntuacion, fechaEstreno, generos[], duracion, reparto[], director, idiomaOriginal

usuarios/{uid}/vistas/{movieId} → (misma estructura)

usuarios/{uid}/pendientes/{movieId} → (misma estructura)

usuarios/{uid}/valoraciones/{movieId} → identificador, titulo, urlCartel, puntuacion (1-5), fechaValoracion

usuarios/{uid}/resenas/{movieId} → identificador, titulo, urlCartel, texto, fecha

usuarios/{uid}/listas_personalizadas/{id} → nombre, emoji, color (int ARGB), peliculas[], fechaCreacion

ranking/{movieId} → { votos: { uid1: 4, uid2: 5, uid3: 3 } }

Anexo C — Endpoints TMDb utilizados

Endpoint	Uso en CineAI
/trending/movie/day	Banner + sección Tendencias
/movie/popular?page=N	Sección Más populares (2 páginas aleatorias)
/movie/{id}/recommendations	Sección Recomendadas para ti
/movie/{id}?append_to_response=credits	PantallaDetalle (detalle completo)
/search/movie?query=X	Búsqueda por texto + CineBot
/discover/movie?with_genres=&vote_average.gte=	Filtros avanzados

Anexo D — Géneros TMDb

Acción: 28 | Aventura: 12 | Comedia: 35 | Drama: 18 | Terror: 27 | Ciencia ficción: 878
| Animación: 16 | Romance: 10749 | Thriller: 53 | Crimen: 80

Anexo E — System prompt de CineBot

Eres CineBot, el asistente de inteligencia artificial de la app CineAI. Tu misión es ayudar al usuario a encontrar la película perfecta para cada momento. Responde SIEMPRE en español. Recomienda ÚNICAMENTE películas. Sugiere entre 3 y 5 películas. Al final añade el bloque: CINEAI_PELICULAS:[{"titulo":"Título","anio":YYYY},...]

Anexo F — Requisitos mínimos del sistema

Usuario final: Android 5.0 (API 21) o superior, conexión a internet, 100 MB de espacio libre.

Entorno de desarrollo: Flutter SDK 3.x, Android Studio, clave de API TMDb (gratuita), clave de API OpenAI (de pago), archivo google-services.json del proyecto Firebase cineai-37e5b.

Anexo G — Métricas de calidad del código

Métrica	Valor
Archivos Dart totales	22
Líneas de código (aprox.)	6.000
Pantallas	15
Servicios	5
Método más extenso	ServicioFirebase (~470 líneas)
Cobertura try/catch asíncrono	100%
APK release	51,9 MB
Emulador de pruebas	Pixel 6 API 34 (Android 14)
Dispositivo físico	Xiaomi Redmi Note 5 (Android 9)